Lionael Surya

Selected Projects

SOFTWARE · DATA · SYSTEMS

01 SAKTI 21
02 FRAUDLENS
03 NUTRITION PRIORITY INDEX
04 DEJABOOTH
05 IDX SIGNAL SCANNER
06 ELDERCARE

2026

ON THE WORK IN THIS BOOK

I like the stretch between *a problem someone describes* and software they can actually use.

Most of the projects here started with a concrete, slightly awkward question. How does a payroll team calculate Indonesian income tax for many client companies without a spreadsheet of hand-copied rates? Which provinces should a nutrition budget reach first, in a way an official can defend? What would a phone look like if it were designed around one grandmother instead of the average user?

I try to answer those questions with something that runs. That usually means modelling the domain carefully first, writing the rules down where they can be checked, testing the parts that would be expensive to get wrong, and then being plain about what still isn't done.

Each project in this book is written up the same way: what problem it addresses, what I built, the evidence that it works, the engineering decisions that mattered, and its honest status today. Several are finished; some are release candidates or alphas. I have said which is which.

The screenshots are taken from the software itself, running locally against each repository's own demo or fixture data, unless a caption says otherwise. Test results quoted were re-run while preparing this edition. Where something could not be reproduced, the caption says so.

STUDYING
Data Science, Seneca Polytechnic, Canada

WORKS ACROSS
Full-stack TypeScript · Python data systems · Applied ML · Android

CODE
github.com/suryalionael

WEB
lionael-portfolio.vercel.app

Contents

NO.	PROJECT	DISCIPLINE	STATUS	PAGE
01	**Sakti 21** Multi-tenant PPh 21 payroll tax system	Full-stack software	Release candidate	04
02	**FraudLens** Explainable transaction risk scoring	ML systems · Data engineering	Complete	07
03	**Nutrition Priority Index** Province prioritisation for child nutrition, Indonesia	Policy analytics · Geospatial	Live	09
04	**Dejabooth** Browser photobooth with client-side rendering	Frontend · Media	Pre-release	11
05	**IDX Signal Scanner** Rules → ML → LLM screener for Indonesian equities	Applied ML · Automation	In daily use	13
06	**ElderCare** Accessibility-first Android companion app	Android · Accessibility	Alpha	14
–	***Also built*** Pipelines, analytics and smaller products			15

READING THE STATUS COLUMN

Complete — the planned scope is built and tested. **Release candidate** — passed its own release audit; not yet deployed for real users. **Live** — publicly deployed. **Pre-release** and **Alpha** — working builds with documented gaps.

01 Sakti 21

A multi-tenant payroll system that calculates Indonesian employee income tax (PPh 21) for many client companies from one application.

STATUS	ROLE	STACK	SOURCE
Release candidate (rebuild). Original version in use across 64 companies.	Sole developer — specification, tax research, implementation, QA	TypeScript · Next.js · PostgreSQL · Prisma · NextAuth · decimal.js · react-pdf · Vitest	Private repository — access on request

THE PROBLEM

Sakti 21 began as a no-code Glide app that a payroll team used for 64 client companies. It worked, but every company shared one password, tax rates were hard-coded into formulas, and a 164-column calculation sheet was the only description of how a payslip figure came to be.

The rebuild had two jobs: keep every number the original produced correct, and make the system defensible — real per-user accounts, tenant isolation enforced in code, and tax rules that are versioned data instead of constants.



FIG. 1.1 A December row resolves to the annual Pasal 17 method and reconciles against the tax already withheld January–November. Synthetic UAT employee.

THE BUILD

I started by reverse-engineering the original app into a written specification: its data model, every tax formula, every screen and automation. That document became the reference the rest of the code is tested against.

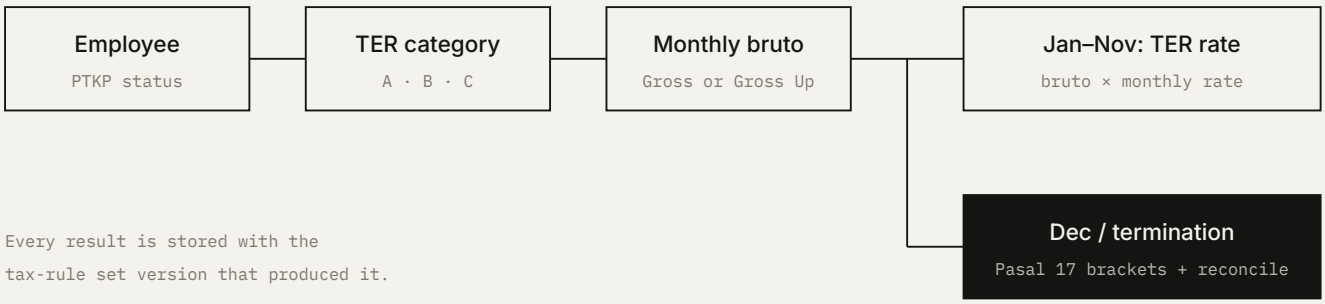


FIG. 1.2 Calculation pipeline, from the engine specification (docs/tax/tax-engine-overview.md).

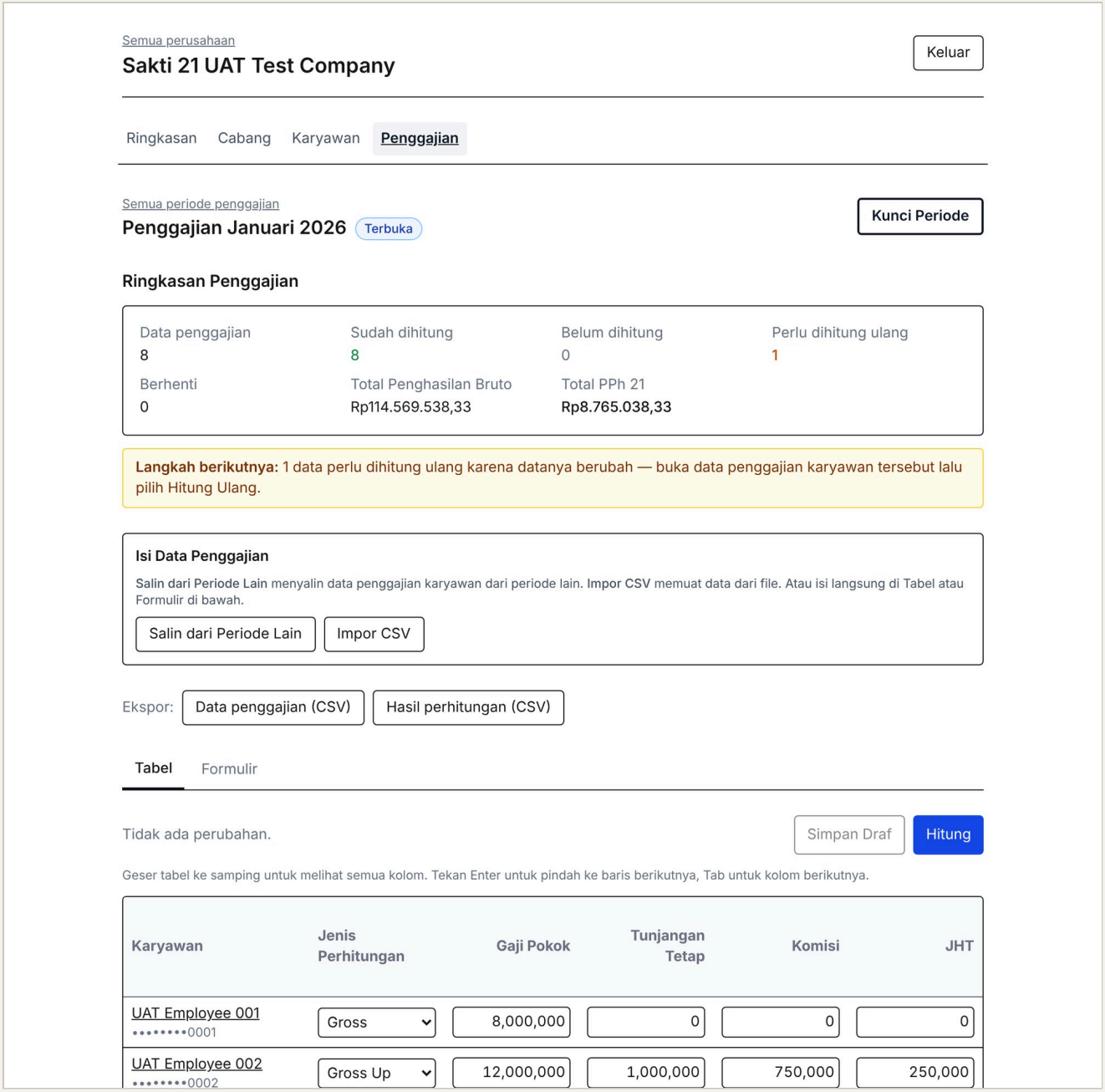


FIG. 1.3 A payroll period: summary counts, a flag for one input that changed after it was calculated, CSV import and export, and an editable grid for monthly inputs. Running locally on the repository's UAT fixture.

WHAT THE OPERATOR DOES

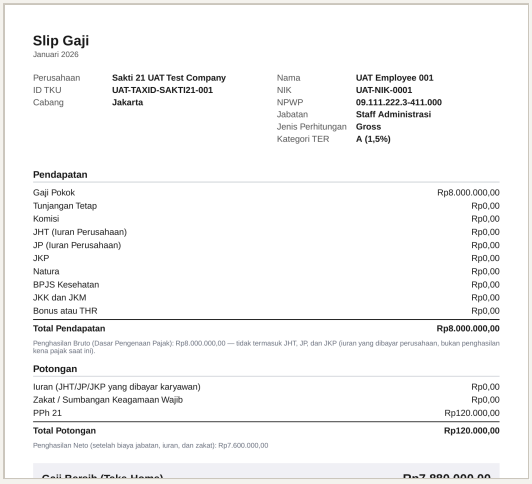
For each client company, staff manage employees (with CSV import), open a monthly payroll period, load salary data by CSV or by hand, and calculate. The app produces monthly and annual reports, CSV exports, and a payslip on screen or as a server-generated PDF.

CORRECTNESS, MADE VISIBLE

Calculations are never silently rewritten. If an input changes after it was calculated — or a December reconciliation depends on a month that changed — the row is marked stale and the period summary says so ("Perlu dihitung ulang: 1").

A DELIBERATE TRADE-OFF

An automatic cascade recompute would quietly change figures that may already have been paid out. Visible staleness is the safer failure — one of 14 architecture decision records, each with the alternatives it rejected.



Slip Gaji
Januari 2026

Perusahaan	**Sakti 21 UAT Test Company**	Nama	**UAT Employee 001**
ID TKU	**UAT-TAXID-SAKTI21-001**	NIK	**UAT-NIK-0001**
Cabang	**Jakarta**	NPWP	**09.111.222.3-411.000**
		Jabatan	**Staff Administrasi**
		Jenis Perhitungan	**Gross**
		Kategori TER	**A (1,5%)**

Pendapatan

Gaji Pokok	Rp8.000.000,00
Tunjangan Tetap	Rp0,00
Komisi	Rp0,00
JHT (Iuran Perusahaan)	Rp0,00
JP (Iuran Perusahaan)	Rp0,00
JKP	Rp0,00
Natura	Rp0,00
BPJS Kesehatan	Rp0,00
JKK dan JKM	Rp0,00
Bonus atau THR	Rp0,00
Total Pendapatan	**Rp8.000.000,00**

Penghasilan Bruto (Dasar Pengenaan Pajak): Rp8.000.000,00 — tidak termasuk JHT, JP, dan JKP (iuran yang dibayar perusahaan, bukan penghasilan kena pajak saat ini).

Potongan

Iuran (JHT/JP/JKP yang dibayar karyawan)	Rp0,00
Zakat / Sumbangan Keagamaan Wajib	Rp0,00
PPh 21	Rp120.000,00
Total Potongan	**Rp120.000,00**

Penghasilan Neto (setelah biaya jabatan, iuran, dan zakat): Rp7.600.000,00

Gaji Bersih (Take Home) **Rp7.880.000,00**

FIG. 1.4 Payslip PDF downloaded from the running app. Rp8,000,000 × the 1.5% TER A rate = Rp120,000 withheld.

THREE DECISIONS THAT MATTERED

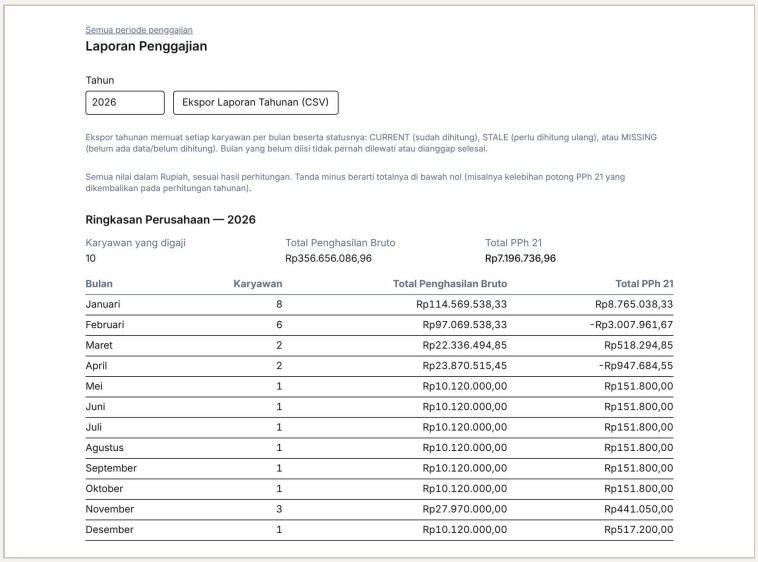
01 **Money is never a float.** Amounts are `numeric(18,2)` in Postgres and `Decimal` in TypeScript end to end. Rounding rules from the regulation — such as flooring taxable income to the nearest Rp1,000 — live in one small module with their own tests.

02 **Tax rules are data, with versions.** PTKP allowances, the 125 TER rate rows and the five Pasal 17 brackets are seeded as a `TaxRuleSet`. Every calculation records which rule set produced it, so a future rate change cannot alter a historical payslip.

03 **The tax engine is pure.** No database, no clock, no framework imports — rule set and inputs in, result out. Gross Up is solved algebraically and re-checked iteratively in case the grossed-up salary crosses into another TER bracket.

OUTCOME

All planned phases are implemented, and the project closed with a release-candidate audit: every one of 42 API routes enumerated, a hostile-ID sweep across tenants over real HTTP, and a feature-parity matrix against the original app. The remaining gaps — two unsourced e-Bupot export fields, payslip layout fidelity, historical data migration — are documented rather than guessed.



Semua periode penggajian

Laporan Penggajian

Tahun

2026 | Ekspor Laporan Tahunan (CSV)

Ekspor tahunan memuat setiap karyawan per bulan beserta statusnya: CURRENT (sudah dihitung), STALE (perlu dihitung ulang), atau MISSING (belum ada data/belum dihitung). Bulan yang belum diisi tidak pernah dilewati atau dianggap selesai.

Semua nilai dalam Rupiah, sesuai hasil perhitungan. Tanda minus berarti totalnya di bawah nol (misalnya kelebihan potong PPh 21 yang dikembalikan pada perhitungan tahunan).

Ringkasan Perusahaan — 2026

Karyawan yang digaji	Total Penghasilan Bruto	Total PPh 21
10	Rp356.656.086,96	Rp7.196.736,96

Bulan	Karyawan	Total Penghasilan Bruto	Total PPh 21
Januari	8	Rp114.569.538,33	Rp8.765.038,33
Februari	6	Rp97.069.538,33	-Rp3.007.961,67
Maret	2	Rp22.336.494,85	Rp518.294,85
April	2	Rp23.870.515,45	-Rp947.684,55
Mei	1	Rp10.120.000,00	Rp151.800,00
Juni	1	Rp10.120.000,00	Rp151.800,00
Juli	1	Rp10.120.000,00	Rp151.800,00
Agustus	1	Rp10.120.000,00	Rp151.800,00
September	1	Rp10.120.000,00	Rp151.800,00
Oktober	1	Rp10.120.000,00	Rp151.800,00
November	3	Rp27.970.000,00	Rp441.050,00
Desember	1	Rp10.120.000,00	Rp517.200,00

FIG. 1.5 Annual report. Negative months are over-withholding returned in a termination reconciliation — shown as such, not hidden.

201

tax-engine and rule-table tests — all passing on a fresh re-run for this edition

709

tests in the full Vitest suite, including tenant-isolation tests against a real database

> On a fresh database with the UAT fixture, one independent-audit assertion (a termination case, UAT-NIK-0008) currently fails. It is being investigated, not hidden.

02 FraudLens

A real-time transaction scoring service that tells a fraud analyst which transactions to look at first, and why.

STATUS	ROLE	STACK	SOURCE
Complete — 139/139 tests passing	Sole developer	Python · FastAPI · PostgreSQL · dbt · scikit-learn · XGBoost · SHAP · Streamlit · Docker · Terraform (AWS ECS, RDS, S3)	github.com/suryalionael/fraudlens

THE PROBLEM

Fraud teams cannot review every transaction, so the useful question is not “is this fraud?” but “which ones first?”. The answer has to be fast, explainable to an analyst, and computed only from information that existed at the moment the transaction happened.

THE BUILD

A transaction arrives at a FastAPI endpoint carrying only raw fields. The server builds behavioural features from history in PostgreSQL, scores with a trained model, explains the score with SHAP, blends it with deterministic rules, and persists the result before responding.

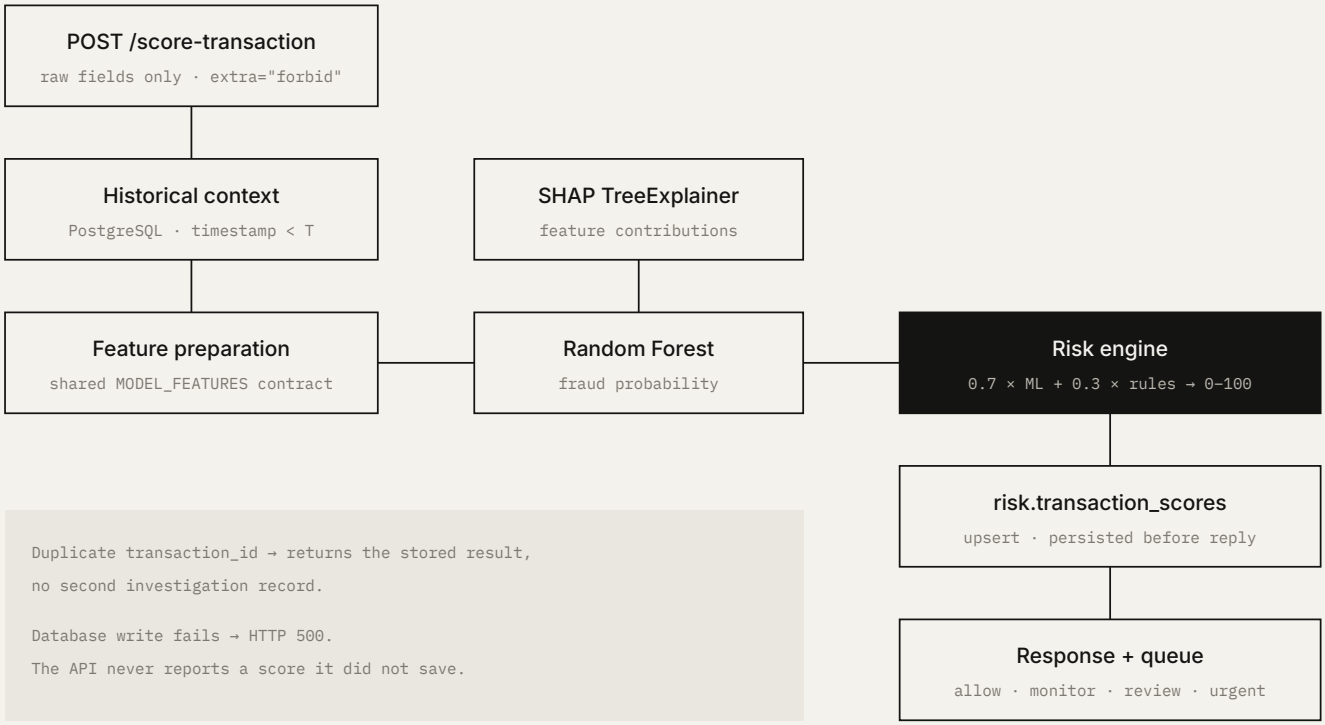


FIG. 2.1 Scoring path as implemented in `src/fraudlens/api/app.py` and `risk/`. Thresholds: review at 60, urgent review at 80.

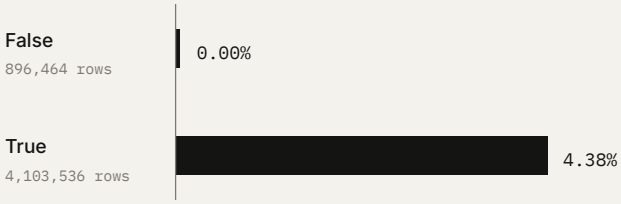
THE FINDING THAT SHAPED THE PROJECT

The dataset's most predictive column was the answer key.

FraudLens trains on a public, synthetic set of five million Nigerian financial transactions (3.59% fraud). Before modelling, I profiled every column against the label. One precomputed field, `new_device_transaction`, was true for every single fraud case. A model would have scored near-perfectly — and learned nothing.

The other "behavioural" columns that came with the data were no better: one had 41% negative time gaps, two were uniformly random with identical fraud rates in every bin. All five are now rejected at the API boundary. Features are rebuilt server-side from raw history, so training and live scoring use exactly the same definitions.

Fraud rate by `new_device_transaction`



Figures from the project's dataset profiling (`docs/dataset-selection.md`). 100% of the 179,553 fraud cases sit in the "True" row.

PRECOMPUTED FIELD	WHAT PROFILING SHOWED
`new_device_transaction`	Target leakage
`time_since_last_transaction`	Broken — 41% negative values
`velocity_score`	No signal — uniform 1–20
`geo_anomaly_score`	No signal — uniform 0–1
`spending_deviation_score`	Negligible signal

```
# risk/historical.py — every context query is cut off at the
# transaction's own timestamp, so no future data leaks in.
SELECT
  COUNT(CASE WHEN timestamp >= %s - interval '10 minutes'
             AND timestamp < %s THEN 1 END) AS last_10m,
  COUNT(CASE WHEN timestamp >= %s - interval '1 hour'
             AND timestamp < %s THEN 1 END) AS last_1h,
  COUNT(CASE WHEN timestamp >= %s - interval '24 hours'
             AND timestamp < %s THEN 1 END) AS last_24h
FROM raw.transactions
WHERE sender_account = %s
```

OUTCOME

A working service with ingestion (schema validation and checksums), a dbt layer, temporal train/test splits evaluated on PR-AUC and precision-at-K, a five-page Streamlit investigation dashboard, and Terraform for AWS. All 139 tests — unit, API, temporal-leakage and idempotency — pass against a real PostgreSQL.

> Limits, as the README states them: synthetic data, no drift monitoring, no API authentication, latency not load-tested. The 5M-row dataset could not be downloaded in the environment used for this edition, so no dashboard screenshots are shown.

03 Nutrition Priority Index

A transparent ranking of Indonesia's 38 provinces for child-nutrition intervention, built only from government data and validated before it was published.

STATUS	ROLE	STACK	LINKS
Live public dashboard · 67/67 tests passing	Sole analyst and developer	Python · pandas · scikit-learn · GeoPandas · PySAL / esda · Streamlit · pytest · GitHub Actions	Live dashboard · Source



FIG. 3.1 Priority score by province, 0 (light) to 1 (black). The five darkest provinces are all in the Papua region. Re-rendered in greyscale for this edition from the project's `npi_spatial.geojson`.

THE PROBLEM

National programmes such as free school meals (*Makan Bergizi Gratis*) have finite budgets and 38 provinces with very different needs. A ranking an official can't audit is a ranking they can't defend — so every source, transformation and modelling choice had to be traceable.

THE BUILD

Seven datasets from Statistics Indonesia's WebAPI, the health ministry and official boundaries, ingested with generated provenance (URLs, dates, checksums). The index combines socioeconomic vulnerability and education access; stunting is kept out of the inputs and used only to validate the result.

THE DECISION I'M PROUDEST OF

The more sophisticated method lost, so it was demoted.

The textbook approach was a PCA composite of three correlated indicators. I tested it head-to-head against a single one — household expenditure per capita. The simple indicator predicted real stunting outcomes slightly better, needs no fitted parameters and explains itself in a sentence. It became the primary input; PCA stays in the project as a documented benchmark.

The robustness work is equally candid. Ranks barely move when the weights shift ±20%, but the Jenks tier labels do — so the dashboard tells users to trust ranks over tier boundaries.

	PCA COMPOSITE	EXPENDITURE PER CAPITA
Spearman r with stunting	0.72	**0.74**
Fitted parameters	Loadings	**None**
Decision-matrix score	13 / 25	**23 / 25**

0.622

Global Moran's I (p = 0.001): need clusters geographically

2.5×

mean priority, Eastern vs. Western Indonesia (ANOVA p = 2.8×10^{-8})

0.67

correlation once partial-coverage provinces are excluded — also reported

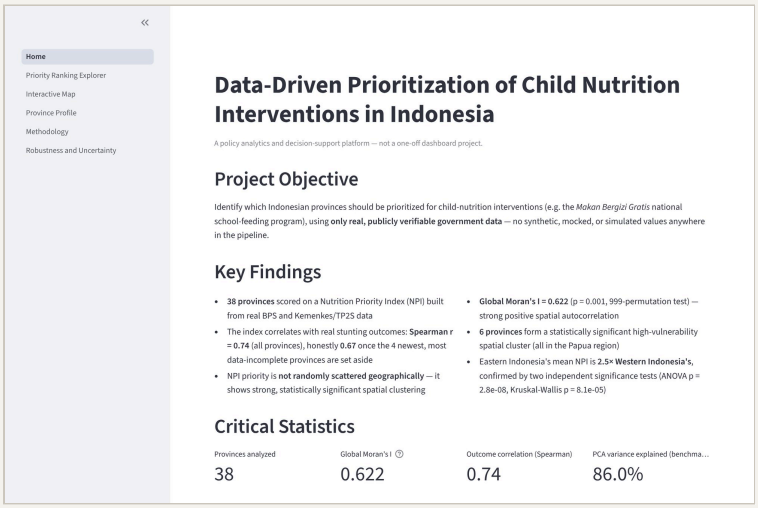


FIG. 3.2 The dashboard's overview page, running locally from the repository's committed data. Six pages in total.



FIG. 3.3 Local spatial clusters (LISA, p < 0.05). Black: six high-need provinces clustered around Papua. Hatched: eight low-need provinces in western Java and Sumatra.

Rank	Province	Percentile	NPI Score	Priority Tier	Data Completeness	Stunting Rate (%)
1	Papua Pegunungan	100.0	1.000	Critical	partial: education_acces	40.0
2	Papua Tengah	97.4	0.853	Critical	partial: education_acces	32.5
3	Papua	94.7	0.815	Critical	full	24.7
4	Papua Barat Daya	92.1	0.783	Critical	partial: education_acces	30.5
5	Papua Selatan	89.5	0.722	Critical	partial: education_acces	25.8
6	Nusa Tenggara Timur	86.8	0.438	High	full	37.0
7	Papua Barat	84.2	0.432	High	full	24.6
8	Maluku Utara	81.6	0.393	High	full	23.2
9	Sulawesi Barat	78.9	0.388	High	full	35.4
10	Sulawesi Tengah	76.3	0.373	High	full	26.1

FIG. 3.4 Ranking explorer: searchable, filterable by tier, with a data-completeness column so readers can see which provinces rest on partial data.

04

Dejabooth

A photobooth that runs entirely in the browser: four shots, a printed-feeling strip, and an animated version — no server involved.

STATUS

Pre-release. 538/538 unit tests; real-device QA outstanding

ROLE

Sole developer

STACK

Next.js · TypeScript · Canvas 2D · WebCodecs · mp4-muxer · IndexedDB · Supabase (optional) · Vitest · Playwright

SOURCE

Private repository — access on request

THE PROBLEM

Photobooth web apps tend to be either a camera with a frame on top, or a pile of separate modes. I wanted one calm, linear ritual — step in, four photos, choose a paper, watch it print, collect it — with outputs good enough to keep.

THE BUILD

Everything is client-side. The app records the session, renders the strip on a canvas, encodes MP4 with WebCodecs and writes real GIF and ZIP files in the browser. Accounts and a cloud gallery exist behind Supabase, but the booth runs fully in guest mode without them.

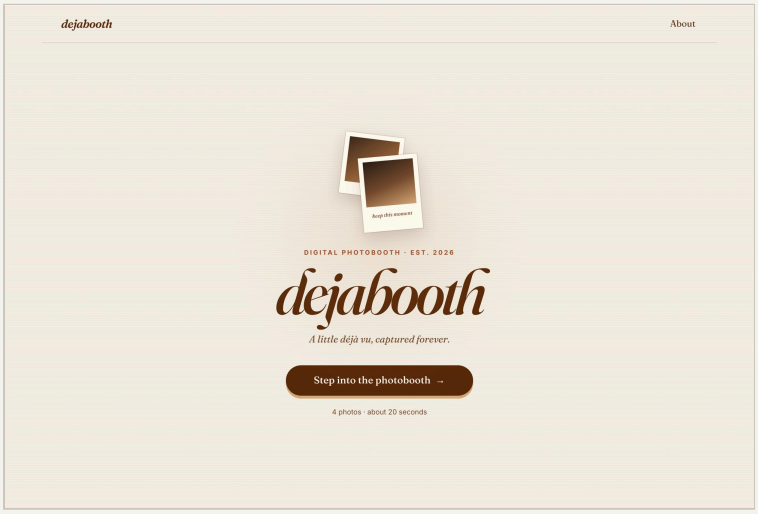


FIG. 4.1 Landing page, running locally.

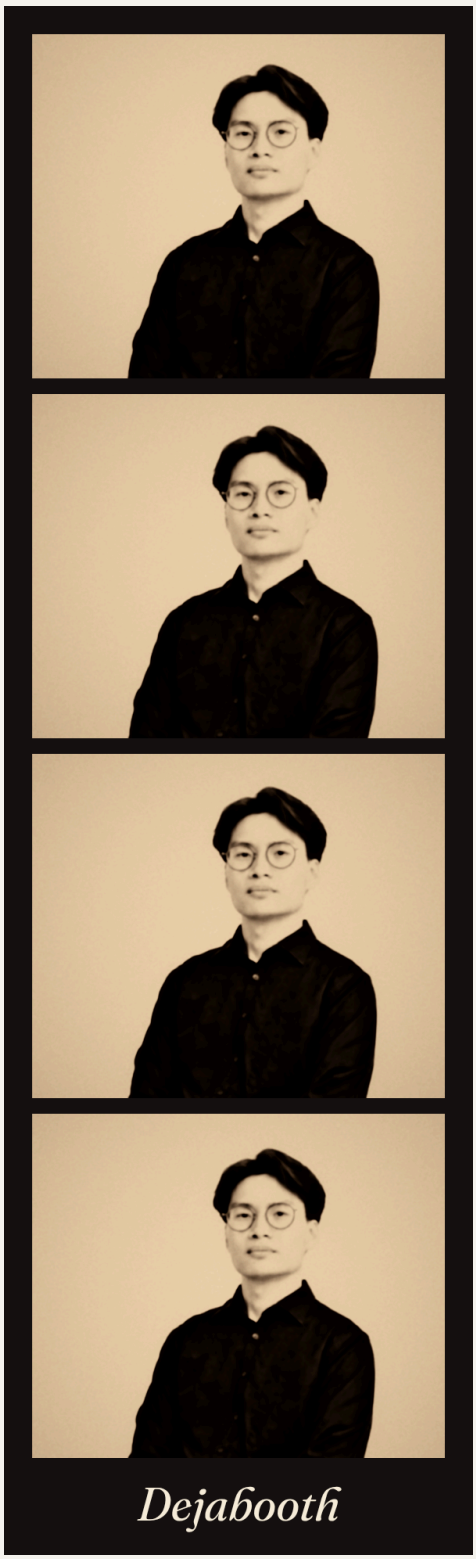
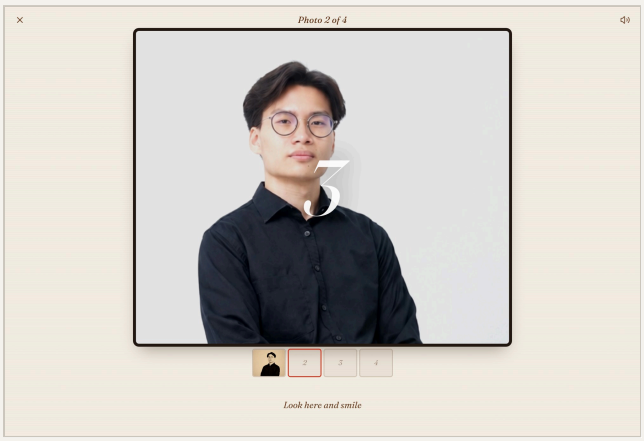
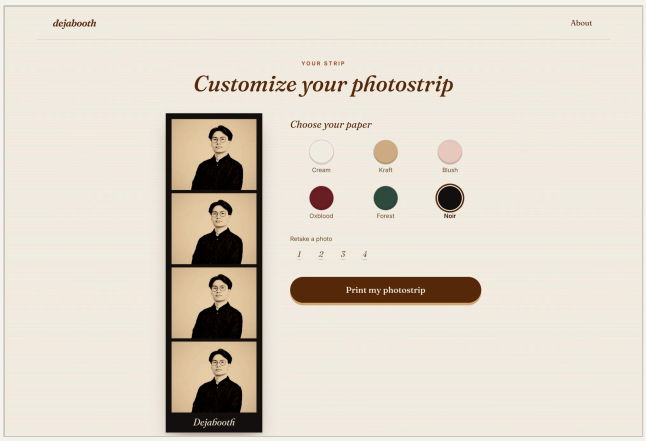


FIG. 4.2 The actual exported 600×1980 PNG, noir paper. The camera was simulated in Chromium using a still portrait, so the four frames match.

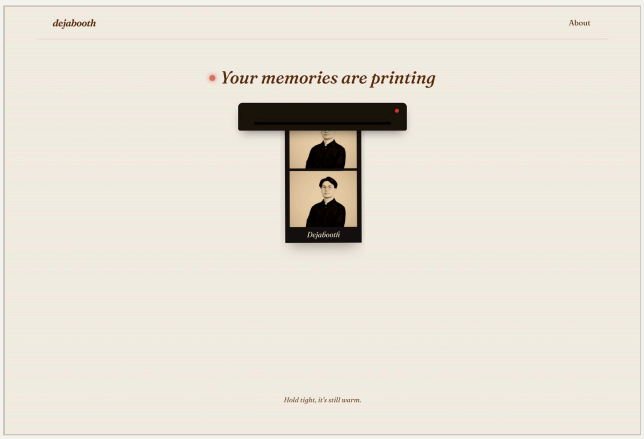
ONE SESSION, START TO FINISH



a Countdown, photo 2 of 4



b Paper colour is the only choice



c The strip feeds out in uneven bursts



d Strip, animated strip and GIF to collect

FIG. 4.3 Captured with Playwright against a local dev server; Chromium's fake camera was fed a still portrait. Screens are unedited.

ONE MODEL FOR EVERY PHOTO

The domain revolves around a single *Photo Set*: ordered slots, each filled by camera or upload. Review, retakes, rendering and export never ask where a photo came from — which removed a whole class of "upload mode" bugs.

MP4 WITHOUT FFMPEG

Most browsers record WebM. Rather than ship a multi-megabyte ffmpeg.wasm, the app encodes H.264 with the native WebCodecs API and a 155 KB muxer — and shows an honest "unavailable" state where a browser can't, instead of mislabelling a file.

ONE EXPOSURE PER PHOTO

The warm monochrome treatment measures exposure from each still once, then applies it everywhere — print, preview, every animated frame, the GIF — so the animated strip can't flicker or drift from the printed one.

> Status, as recorded in the project: unit tests, type checks, a production build and end-to-end runs in desktop Chrome and WebKit (iPhone emulation) pass. It has not yet been tested on a physical phone, and it is not deployed.

05 IDX Signal Scanner

A daily screener for the Indonesian stock exchange where rules filter, a model ranks, and a language model explains.

STATUS

In daily use as a personal research tool. Decision support — explicitly not advice.

ROLE

Sole developer

STACK

Python · pandas · Parquet · XGBoost · Claude API · Streamlit · pytest

SOURCE

github.com/suryalionael/idx-stock-scanner-agent

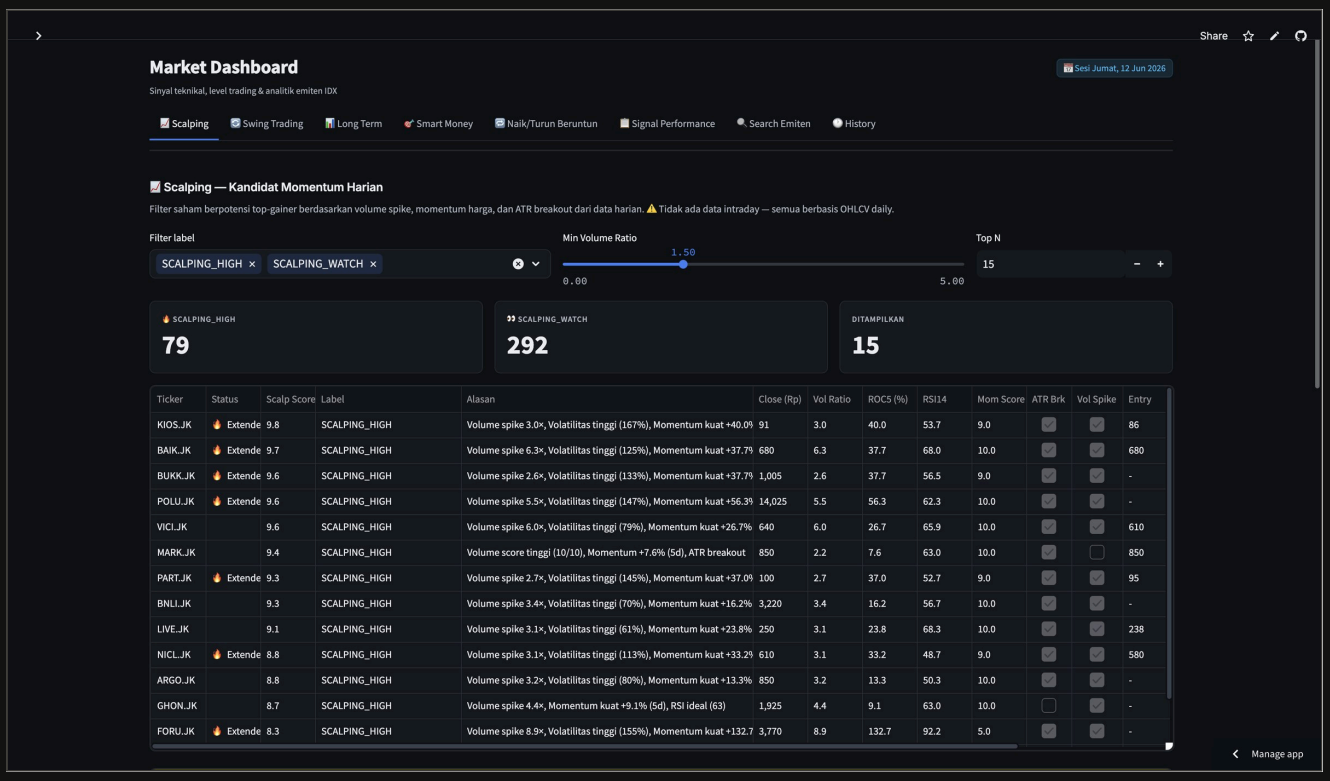


FIG. 5.1 Market dashboard, 12 June 2026 session. Screenshot taken by me at the time; the latest data bundle was not available for a fresh capture.

EACH LAYER DOES ONE JOB

Deterministic rules reject illiquid or broken tickers first, so every rejection has a stateable reason. An XGBoost ranker then orders the survivors; an LLM writes a short note on why a signal fired. The LLM never picks.

An audit found the original rules ranked 248 stocks on one day — including a #1 "breakout" with Rp117M of turnover. A hard liquidity gate and re-weighted penalties cut that to 64.

RANKER, OUT-OF-TIME TEST · NOV 2025 – MAY 2026

AUC-ROC	0.594	
Precision when it flags	0.505	vs. 0.265 base rate
Recall	0.028	
Rows (train / test)	204,436 / 51,109	

From `models/ranker_metrics.yaml`. A weak but real signal: right about half the time when confident, almost never confident. That is why it ranks after the rules rather than deciding on its own.

06 ElderCare

An Android companion app built for one specific person — my grandmother — around a single rule: dignity over surveillance.

STATUS	ROLE	STACK	SOURCE
Alpha (v0.3, Sprint 5 in progress). Offline core complete; family dashboard not yet built	Sole developer	Kotlin · Jetpack Compose · Room · WorkManager · Android TextToSpeech	Private repository — access on request

THE PROBLEM

Stock Android assumes small text, steady hands and familiarity with icons. Commercial “senior phones” are tied to hardware or carriers. And most family-monitoring apps quietly watch the user. This one must only do things she can see happening.

THE BUILD

A home screen with the time, the next reminder and large tiles; local reminders scheduled with WorkManager that arrive as full-screen alerts and are read aloud offline; one-tap calls to family contacts; a settings screen for text size, contacts and reminders. Everything works with no account and no network.

ACCESSIBILITY, AS TESTS

Primary actions are 88 dp — nearly twice Android’s minimum. The palette’s WCAG AA contrast ratios are asserted in unit tests, so a colour change that breaks them fails the build. A UX review at 190% text size found the text-size setting had never actually applied; the fix scales the whole type system from one place.

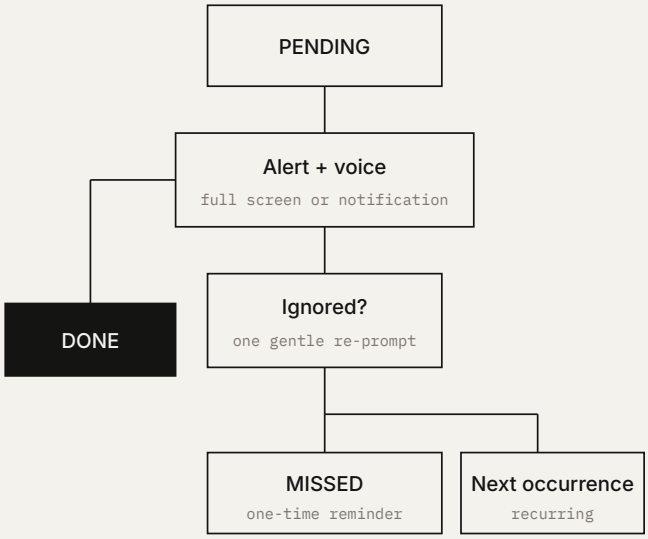


FIG. 6.1 Reminder lifecycle, from `ReminderCompletion.kt` and `OverdueReminderWorker`. An ignored recurring reminder never kills the series.

```
// ThemeContrastTest.kt — WCAG 2.2 AA as a unit test
@Test
fun emergencyOnWhite_meetsBodyAndLargeAA() {
    assertContrast(emergency, background, 4.5)
}

@Test
fun whiteOnPrimary_meetsLargeAA() {
    assertContrast(white, primary, 3.0)
}
```

> The repository holds 121 JVM unit tests and 36 instrumented Compose tests. Device screenshots and test runs need the Android SDK, which was not available for this edition — so this chapter shows code and structure instead.

Also built

Smaller or earlier projects, described as their repositories describe them.
Not re-verified to the same depth as the six chapters.

DATA ENGINEERING

Streaming Clickstream Pipeline

Kafka → Spark Structured Streaming → Delta Lake (bronze/silver/gold on MinIO) → DuckDB → Streamlit, with watermarking, a dead-letter queue and a seven-service Docker Compose stack.

SOURCE

DATA ENGINEERING

Real-Time F1 Analytics Platform

The same medallion pattern applied to Formula 1 race data, served through a FastAPI backend to a React + Recharts dashboard, with LLM-written race summaries.

PRIVATE REPOSITORY

DATA ENGINEERING

Retail Sales ETL Pipeline

Batch ETL over the UCI Online Retail data: validation at the boundary, a star-schema warehouse in PostgreSQL, Airflow orchestration and idempotent re-runs.

SOURCE

ANALYTICS ENGINEERING

dbt E-Commerce Analytics Mart

Staging → intermediate → marts in dbt on PostgreSQL, with tests and documentation, feeding customer, finance and core models.

SOURCE

MACHINE LEARNING

Customer Churn Prediction

Telecom churn model with calibrated probabilities, SHAP explanations, ROI-based thresholding, FastAPI and Streamlit serving, and drift monitoring.

PRIVATE REPOSITORY

MACHINE LEARNING

Retail Demand Forecasting

Forecasting with a backtesting framework comparing statistical and gradient-boosted models.

SOURCE

FULL-STACK

Aspen OS

A project operating system for nonprofit and community organisations — tasks, kanban and documentation — on Next.js and Supabase.

SOURCE

AUTOMATION

Employee CV Generator

Google Apps Script tool that generates standardised CVs from Sheets data for an HR team at PT Magna Solusi Indonesia.

PRIVATE REPOSITORY

CLOUD · DESIGN PHASE

CloudLake

Architecture for a serverless AWS retail data platform (Lambda, Glue, Athena, dbt, Terraform). Documentation complete; no resources deployed yet.

SOURCE

LIONAEL SURYA

Thank you for reading.

GITHUB	github.com/suryalionael
PORTFOLIO	lionael-portfolio.vercel.app
EMAIL	suryalionael@gmail.com

COLOPHON

Set in Newsreader, Inter and IBM Plex Mono. Typeset in HTML and CSS and printed to PDF with headless Chromium. Screenshots were captured with Playwright from local runs of each application on its own demo or fixture data; figures marked otherwise are noted in their captions. Several projects were developed with AI coding assistants under my direction; the repositories document how.

EDITION 2026 · OCTOBER